Technical Instructions for Working With Hebrew Files

Searching Hebrew Text: `./claude/perush-tools/hebrew-grep`

Standard grep cannot search Hebrew in this project — niqqud (vowel marks like בְּרֵאשִׁית) and non-Hebrew characters (backticks, markdown) break pattern matching. **Always use `./claude/perush-tools/hebrew-grep` instead of Grep/grep/rg for Hebrew text searches.**

How it works: Before matching, every line is normalized — niqqud is stripped and all non-Hebrew sequences collapse to a single space. The search sees only space-delimited bare Hebrew words. Spaces are added at line boundaries, so `'נחש'` matches the whole word.
The same normalization is applied to the pattern (so niqqud in the pattern is ignored too), plus:

- **Final letters** are treated as regular letters (`כמנפצ` = `רמוקיץ`): `'ארץ'` also finds `ארצה`, `ארצות` - word-ends are already marked by the spaces.
- **Shin**: a plain `ש` matches every Shin (undotted, `שׁ` and `שׂ`), while a dotted `שׁ` / `שׂ` matches only that dot - so it only finds vocalized text (the biblical verses), not the unvocalized commentary. Precomposed and decomposed Unicode forms are the same. Character classes that include `ש` (also as part of a range like `[א-ת]`) cover all its forms.

Usage: `./claude/perush-tools/hebrew-grep <JS RegExp> <files-or-folders...>`
Folders are searched recursively (skipping temporary local variants of commentary files, named `*.<number>.rtl.md`). Output is YAML-like: file path, then matching lines with line numbers and verse references.

Examples:

```
# Find exact word אדמה across all commentary
./claude/perush-tools/hebrew-grep ' פירוש ' אדמה/

# Find the word תרבות with any prefix in a specific file (will also find והתרבות
./claude/perush-tools/hebrew-grep 'בראשית-א_א-ב_ג-שבע_ימי_הבריאה/בראשית-1010/בראשית-1/ש

# Find any word with "נ" followed by "ח" followed by "ש" in two files
./claude/perush-tools/hebrew-grep 'בראשית-ב_ד-ב_יז-גן_עדן/בראשית-1020/בראשית-1/פירוש/ ' ש

# Find the verb עשׂה (Sin) in the verses - excluding words with Shin, like עשׁן
./claude/perush-tools/hebrew-grep ' בראשית-1/פירוש/ עשׂ'
```

Finding the Commentary Files of Verses: `./claude/perush-tools/files-of-verses`

Lists the paths of the commentary files (under `./פירוש/`) that cover the referenced verses - one path per line, in Torah order.
Chapters and verses are Hebrew numerals (`ד`, `יב`, `טו`) or digits; quoting is optional.
A reference that no file covers (e.g. a verse that doesn't exist) is an error (exit code 1).

```
./claude/perush-tools/files-of-verses בראשית ד:ז              # a single verse
./claude/perush-tools/files-of-verses בראשית ד:א-ה            # verses 1-5 of ch
./claude/perush-tools/files-of-verses בראשית ד:א-ה:ג          # from 4:1 to 5:3
./claude/perush-tools/files-of-verses בראשית ד                # the whole chapte
./claude/perush-tools/files-of-verses בראשית                  # the whole book
./claude/perush-tools/files-of-verses בראשית ד:א שמות ב:ג     # from the first r
```

Editing Hebrew Files: Niqqud and Combining-Mark Order

The Edit tool can fail with `String to replace not found` on Hebrew text that looks character-for-character identical to what Read returned. The cause is **Unicode combining-mark order**: niqqud (qamatz `ָ`, dagesh `ּ`, etc.) are combining marks, and the same visible word can be encoded with marks in different byte orders (e.g., letter+qamatz+dagesh vs. letter+dagesh+qamatz). Edit does exact byte matching and rejects strings that *look* right but differ in mark order. The error message is unhelpfully generic.

If Edit fails on Hebrew text with niqqud — do NOT retry with a re-typed `old_string`. Re-typing reproduces the same wrong byte order. Switch to Python via Bash:

1. Locate the target block using **niqqud-free anchors** — ASCII or unmarked Hebrew (e.g., `כל -'` `'כל בהמה ובהמה"`, `'.='`). These match reliably regardless of mark order.
2. Extract the block verbatim from the file with `content[start:end]` — don't retype the niqqud-bearing portion.
3. Build the replacement (new niqqud you write is fine — its mark order needn't match anything pre-existing).
4. `content.replace(old_block, new_block, 1)` and write back.

```
path = '...'
with open(path, 'r', encoding='utf-8') as f:
    content = f.read()
start = content.find('<niqqud-free start anchor>')
end_anchor = '<niqqud-free end anchor>'
end = content.find(end_anchor) + len(end_anchor)
old_block = content[start:end]
new_block = """..."""
assert content.count(old_block) == 1
content = content.replace(old_block, new_block, 1)
with open(path, 'w', encoding='utf-8') as f:
    f.write(content)
```

Diagnostic when two Hebrew strings look identical but don't match — find the first byte-level divergence:

```
for i, (a, b) in enumerate(zip(actual, target)):
    if a != b:
        print(f"Diff at offset {i}: file={hex(ord(a))}, target={hex(ord(b))}")
        break
```